

Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages

Darwin Morris

June 2026

1 Introduction

This project is based on the paper *Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages* by Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan.

The paper looks at the problem of embedding a typed object language inside a typed host language. Traditionally one sees object programs embedded as abstract syntax trees and interpreted using a universal value type. This approach introduces runtime overhead through pattern matching on syntax and tagging of object-language values. Additionally, the host language type system cannot guarantee that the object-language programs are well typed.

Through a family of tagless-final interpreters the paper shows how to eliminate these sources of overhead while maintaining type safety. Rather than representing programs through their initial encodings, as abstract syntax trees, object-language terms are represented directly through their meaning (denotationally). Object-language types are encoded directly as host-language types, allowing for the creation of interpreters that are guaranteed not to get stuck. Using this approach, the paper presents an evaluator, compiler, partial evaluator, and continuation-passing style (CPS) transformers. All of this is done without relying on "fancy" features such as generalized algebraic data types (GADTs) or dependent types.

The goal of this project, other than to understand the paper, was to reimplement the main ideas in Scala 3. I implemented a tagless-final evaluator, compiler, and partial evaluator, and CPS interpreters for the object-language described in the paper, along with some other example code. I chose Scala 3 because it provides higher-kinded types for type constructors, type classes, and other functional programming abstractions. This gave me the correct machinery to follow the papers Haskell and OCaml implementations, while remaining distinct enough to be interesting. The fact that the Scala community so heavily adopted the tagless-final construct also piqued my interest.

The full source code, tests, and build instructions for the project are available at:

<https://github.com/darwinmorris/finally-tagless-scala>

2 Implementation

2.1 Initial Encoding and the Tag Problem

Before implementing the tagless-final approach, I first reproduced the initial encoding presented in Section 1.1 of the paper. The simplified object-language consists of booleans, variables represented using de Bruijn indices, lambda abstractions, and applications.

In Scala 3, the syntax of the object language can be represented, in familiar initial style, using algebraic data types:

```
enum Var:
  case VZ
  case VS(v: Var)

enum Exp:
  case V(v: Var)
  case B(b: Boolean)
  case L(body: Exp)
  case A(fun: Exp, arg: Exp)
```

Now an object term such as $(\lambda x.x) \text{ true}$ can be expressed by:

```
val test1 = A(L(V(VZ)), B(true))
```

Upon implementing an evaluator the tagging issues immediately become apparent. The object language contains values of different types, such as booleans and functions, but the evaluator itself must return a single metalanguage type. In order to appease the type system we must introduce a universal type:

```
enum U:
  case UB(b: Boolean)
  case UA(f: U => U)
```

The evaluator therefore has the type `eval0 : List[U] => Exp => U`

Now every object-language value is represented as a value of type `U`. This representation allows for interpretation in a statically typed language, but it comes at a cost: type information about object-language expressions is not preserved and must be recovered through runtime tags and pattern matching.

This approach permits the representation of both well-typed and ill-typed object programs. Due to this, operations such as function application require non-exhaustive pattern matching and may fail at runtime. Even though evaluation never fails on well-typed closed terms, this safety is not visible to the metalanguage type system. The inability of the initial encoding (AST) to preserve object-language types is stated as a primary motivation for the tagless-final encoding introduced in the following section.

2.2 A Final Encoding

The paper's first solution to the tag problem involves representing object programs using functions instead of data constructors. Object terms can now be represented by their meaning (denotationally).

The final encoding eliminates the syntax tree of the previous section, trading in algebraic data types for scala functions:

```
def varZA, Env: A = env._1
def varS[A, B, Env](vp: Env => A)(env: (B, Env)): A = vp(env._2)
def b[Env](bv: Boolean): Env => Boolean = _ => bv
def lam[A, B, Env](e: ((A, Env)) => B): Env => (A => B) =
  env => x => e((x, env))
def app[A, B, Env](e1: Env => (A => B), e2: Env => A): Env => B =
  env => e1(env)(e2(env))
```

Using this representation, the object term $(\lambda x.x) \text{ true}$ can now be expressed as:

```
val testf1 = app(lam(varZ), b(true))
```

This is almost the same as in the previous section, except with lowercase ident. The universal type is no longer required, now boolean values are represented directly as Scala booleans and functions are

represented as Scala functions. We are now free of runtime tags and pattern matching. Additionally trying to evaluate an open term with an empty environment produces a compile-time type error rather than a runtime failure.

By representing object-language terms denotationally rather than by their syntax, the safety of the language is now apparent to the metalanguage. This is done without introducing GADTs, dependent types, or intensional type analysis. The rest of the paper generalizes this idea to the tagless-final style, where a family of interpreters, each with the same signature, can evaluate, compile, and partially evaluate the object language.

3 Abstracting the Interpreter

The denotational encoding presented in the previous section successfully eliminates the universal type and runtime pattern matching. However, the evaluator is fixed: every object program can only be interpreted according to the functions `varZ`, `lam`, `app`, and `b`.

The central idea of the tagless-final approach is to abstract over these operations. Rather than directly defining their implementation, the object language is described by an interface whose methods correspond to the language constructs. Different implementations of this interface then provide different interpretations of the same object program.

In Scala 3 this maps to a trait with a higher-kinded type parameter, and given instances of the trait. This is a rather direct translation from the Haskell type class version in the paper.

```
trait Symantics[Repr[_]]:
  def int(n: Int): Repr[Int]
  def bool(b: Boolean): Repr[Boolean]

  def lam[A,B](f: Repr[A] => Repr[B]): Repr[A => B]
  def app[A,B](f: Repr[A => B], x: Repr[A]): Repr[B]
  def fix[A](f: Repr[A] => Repr[A]): Repr[A]

  def add(lhs: Repr[Int], rhs: Repr[Int]): Repr[Int]
  def mul(lhs: Repr[Int], rhs: Repr[Int]): Repr[Int]
  def leq(lhs: Repr[Int], rhs: Repr[Int]): Repr[Boolean]

  def if_[A](cond: Repr[Boolean], thenBranch: => Repr[A], elseBranch: => Repr[A]): Repr[A]
```

The type constructor `Repr[_]` (of kind $\star \rightarrow \star$) represents the semantic domain in which object programs are interpreted. In practice this can be thought of as the type used to represent object-language values (e.g. choosing `Repr[A] = A` constructs an evaluator). This corresponds directly to the `repr` parameter of the `Symantics` type class presented in the paper.

Now programs can be represented as polymorphic expressions that can be interpreted by any implementation of the `Symantics` trait. For example, the object term $(\lambda x.x) \text{ true}$ can be written as:

```
def test1[Repr[_]](using S: Symantics[Repr]) =
  import S.*
  app(lam(x => x), bool(true))
```

The type of `test1` is polymorphic in `Repr`, meaning that the same object program can be evaluated in multiple ways by providing different implementations of `Symantics`.

3.1 Evaluator Interpretation

The simplest implementation of `Symantics` is a direct evaluator. In this interpretation, termed `metacircularRepr`, `Repr` is instantiated as the identity type constructor `Id[A] = A`. As a result of this

object-language values are represented directly by their Scala values:

```
type Id[A] = A

given Symantics[Id] with
  def int(n: Int) = n
  def add(x: Int, y: Int) = x + y
  def lam[A, B](f: Id[A] => Id[B]): Id[A => B] = f
  ...
```

Interpreting `app(lam(x => x), bool(true))` under this instance evaluates directly to the Scala value `true`. No syntax tree is built. This interpreter corresponds to the evaluator `R` presented in the paper and serves as the foundation for the compiler and partial evaluator developed in later sections.

3.2 Compiler Interpretation

The paper demonstrates that compilation can be expressed as another interpreter over the object language. In the MetaOCaml implementation the object-language is compiled into code using MetaOCaml’s staging. As a result, interpreting an object term produces executable code rather than a value.

Scala 3 does not provide the same staging facilities used by the papers MetaOCaml implementation. Instead, I followed the alternative approach described by the authors for their Haskell implementation. Rather than generating staged code directly, the compiler interpretation constructs a typed syntax tree representing the residual program.

The target language of the compiler is represented by the following datatype:

```
enum Code[A]:
  case VarC[A](name: String) extends Code[A]
  case IntC(n: Int) extends Code[Int]
  case BoolC(b: Boolean) extends Code[Boolean]
  case LamC[A, B](param: String, body: Code[B]) extends Code[A => B]
  ...
```

The compiler is implemented as another instance of the `Symantics` type class:

```
given Symantics[Code] with
  def int(n: Int): Code[Int] = Code.IntC(n)
  ...
```

Because `Code` itself is a type constructor of kind $\star \rightarrow \star$, it can be used directly as the representation type `Repr`. The object-language type information is preserved through the type parameter of `Code[A]`. The datatype is represented as a GADT, refining the result type, and ensuring that the constructed programs are well-typed.

Because the compiler must create an explicit syntax tree, lambda and fixpoint require the generation of fresh variable names:

```
def lam[A, B](f: Code[A] => Code[B]): Code[A => B] =
  $val x = fresh(x)
  Code.LamC(x, f(Code.VarC(x)))
```

Interpreting the example term `app(lam(x => x), bool(true))` with the compiler now produces a syntax tree corresponding to the compiled program $(\lambda x. x) \text{ true}$, using the constructors `LamC`, `AppC`, and `BoolC`. The object program again remains unchanged, while only the interpretation of the `Symantics` interface changes.

3.3 Partial Evaluation

The next interpreter I implemented is a tagless partial evaluator, corresponding to Section 4 of the paper. A partial evaluator reduces computations that are known at the current stage, and emits residual code for computations that must be run later. As per the paper, it uses the evaluator to reduce static terms and the compiler to build dynamic terms.

The papers first attempt to classify each term as either static or dynamic runs into the polymorphic lift problem. For example if an `if` expression has a dynamic condition but a static branch, the static branch must be converted into residual code. This would require the lift function $A \Rightarrow \text{Code}[A]$.

The paper avoids this problem by building dynamic code alongside static values. My implementation uses this idea by representing partially evaluated terms as:

```
case class PE[S, D](
  st: Option[S],
  dy: Code[D]
)
```

The `st` component contains a static value when one is known, and the `dy` part always contains the residual code.

Handling functions requires further extension. The previous interpreters used the $\star \rightarrow \star \text{ Repr}[A]$. Partial evaluation needs both a static and dynamic interpretation of each term, so $\star \rightarrow \star \rightarrow \star$. I generalized the representation to:

```
trait SymanticsPE[Repr[_], _]:
  type SFun[A, B] = Repr[A, A] => Repr[B, B]
  type RLam[A, B] = Repr[SFun[A, B], A => B]
  ...
```

The first type parameter now records the static representation while the second records the dynamic. The static representation is now itself a host language function over the same partially evaluated terms. The change in signature of the trait does mean we must lift the representation of the compiler and evaluator from $\star \rightarrow \star$ to $\star \rightarrow \star \rightarrow \star$. Since these interpreters don't make use of the static param it is ignored.

The implementation of `lam` and `app` capture this idea, a lambda stores the HOAS function as its static component while constructing residual code alongside it:

```
def lam[A, B](f: PE[A, A] => PE[B, B]): RLam[A, B] =
  PE(Some(f), ...)
def app[A, B](f: PE[PE[A, A] => PE[B, B], ...], x: PE[A, A]): PE[B, B] = f.st match
  case Some(f) => f(x)
  case None => pdyn(compiler.app(abstr(f), abstr(x)))
```

When a function is applied, the evaluator first checks whether this static function is available. If it is, it gets applied directly to the partially evaluated argument, otherwise an application node is emitted into the residual program. This allows beta-reductions to occur even when arguments are only partially known.

For fixpoint, I implemented the residualizing version described in the paper, avoiding any aggressive unfolding that can lead to nontermination or excessive code growth.

By composing the evaluator and compiler interpretations, we obtain a partial evaluator that performs online binding-time analysis, never gets stuck on well-typed object programs, and produces well-typed residual code.

4 Validation

The implementation was validated using ScalaTest unit tests covering the evaluator, compiler, partial evaluator, and CPS. The tests were primarily derived from examples presented in the paper together with additional cases for simplifications and residualization.

For the partial evaluator, tests verified constant folding, elimination of “neutral elements”: addition by zero and multiplication by one, evaluation of statically known conditionals, beta-reduction of identity functions, and correct residualization of dynamic expressions.

5 Limitations

This project focused on reimplementing the core ideas of the paper rather than reproducing every interpreter presented by the authors exactly.

The original paper uses MetaOCaml to generate staged code directly. Scala 3 does not provide equivalent staging facilities, so the compiler and partial evaluator target a `Code[A]` representation rather than executable generated code. An extension here would be to integrate with a code gen backend so generated programs could be executed directly.

While I implemented the call-by-value and call-by-name CPS interpreters from Section 5, I did not complete a full reproduction of the CPS hierarchy presented in the paper. The report therefore focuses primarily on the evaluator, compiler, and partial evaluator.

5.1 Reflection

Reading this paper and implementing this project gave me a first view into the world of representing embedded programs in a way other than as abstract syntax. The tagless-final style struck me as an elegant alternative, allowing programs to leverage the host language type system to enforce object-language typing, and allowing the program to be interpreted differently by changing its representation. Working on creating a type checker in the AFP class doubly made me appreciate the advantages of leveraging the host compiler as an object-language type checker.

I also was surprised by the flexibility provided by the representation type `Repr`. Because `Repr` is itself a type constructor, interpreters can choose almost any representation that matches the required shape. In my implementation, `Repr` was instantiated as a type alias for direct evaluation, as a typed syntax tree for compilation, and as a two-parameter object representation for partial evaluation.

I did throughout this process sometimes question why this paper was so seminal, the authors themselves claim many times to not have directly invented many of these techniques. Over time, however, I came to appreciate the paper as a very coherent practical synthesis of those ideas. The main challenge was understanding the motivation behind some of the paper’s constructions. While I found the progression from evaluator to compiler to partial evaluator easy to follow, I felt the paper was often overly focused on implementation details rather than the underlying foundational ideas. They heavily reference prior work, that is often largely theoretical, but I felt it was entirely on me to connect those theoretical threads. I often felt that I understood what a particular piece of code was doing before I understood why it was necessary. Oleg Kiselyov’s lecture notes were helpful in filling in some of these gaps, although I still found myself with a sense that I was standing on the surface of something more fundamental.

I found this process immensely fun, frustrating, and insightful. Beyond reproducing the results of the paper, it gave me a better understanding of higher-kinded types, higher-order abstract syntax, partial evaluation, and the role of representations in embedded language design. I guess I have many more trips to <https://okmij.org/ftp/> in my future.

References

1. Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. *Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages*. JFP, 2009.
2. Oleg Kiselyov. *Typed Tagless Final Interpretations*. <https://okmij.org/ftp/tagless-final/course/>
3. N. Rinaudo. *A "Quick" Introduction to Tagless Final*. https://nrinaudo.github.io/articles/tagless_final.html